

[Accueil](#) / [Articles](#) / [CV](#) / [English](#) / [RSS](#)[Tout](#) / [Tech](#) / [Réflexions](#)Publié le **25-06-2021** par *Vincent Jousse*Dernière mise à jour Le **11-02-2024**Tags [#web](#) [#python](#) [#framework](#) [#fastapi-tutorial](#) [#fastapi](#)

Le guide complet du débutant avec FastAPI - Partie 3 : réorganisation du code, tests automatisés

Jusqu'ici nous avons placé tout notre code dans le même fichier `main.py`. Même si nous pourrions continuer comme cela, il est souvent préférable de séparer son code dans des fichiers et des modules différents. Cela va nous aider à nous y retrouver et va encourager le fait de séparer les responsabilités/préoccupations ([Separation of concerns](#) en anglais). Lors de la [partie 2](#) nous avons déjà posé quelques bases en créant des répertoires pour les modèles, les templates, le core, etc. Il est maintenant temps d'aller plus loin et de les utiliser à bon escient.

Mise à jour le 11/02/2024 : Tortoise n'étant pas activement maintenu, j'ai décidé de passer le tutorial de Tortoise ORM à SQL Alchemy

L'intégralité du guide est [disponible ici](#).

Restructuration du code

Connexion à la base de données

Créez un fichier `database.py` dans le répertoire précédemment créé à l'emplacement `app/core`. Nous allons y placer la logique de connexion à

la base de données. Mettez-y le contenu suivant qui était auparavant dans `app/main.py` et supprimez-le de `app/main.py` :

```
python
```

```
# app/core/database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

# DB Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Il vous faudra ensuite mettre à jour les imports dans `app/main.py` pour importer le nécessaire pour la connexion à la base :

```
python
```

```
# app/main.py

from fastapi import Depends, FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.database import Base, engine, get_db
```

Les modèles

Créez un fichier `article.py` dans le répertoire précédemment créé à l'emplacement `app/models`.

Mettez-y le contenu de votre modèle qui se trouvait précédemment dans `main.py`.

python

```
# app/models/article.py

from sqlalchemy import Column, DateTime, Integer, String
from sqlalchemy.sql import func

from app.core.database import Base

class Article(Base):
    __tablename__ = "articles"

    id = Column(Integer, primary_key=True)
    title = Column(String)
    content = Column(String)

    created_at = Column(String, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(),
onupdate=func.now())

    def __str__(self):
        return self.title
```

N'oubliez pas de supprimer les lignes correspondantes dans `main.py`. Il nous faut maintenant importer ce nouveau fichier dans `main.py`.

python

```
from app.models.article import Article
```

L'entête de votre fichier `main.py` devrait ressembler à cela maintenant :

```
python
```

```
# app/main.py

from fastapi import Depends, FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.database import Base, engine, get_db
from app.models.article import Article

# ... reste du fichier
```

Pour être sûr que vous n'avez rien cassé, vous pouvez aller vérifier que [la page listant vos articles](#) fonctionne toujours correctement.

Fichier de configuration

Pour préparer la suite, nous allons avoir besoin de séparer la configuration de notre application dans un module à part. Pour ce faire, créez un fichier `config.py` dans le répertoire `app/core` et placez-y la déclaration de notre objet `templates` :

```
python
```

```
# app/core/config.py

from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="app/templates")
```

Enlevez le code correspondant dans `app/main.py` puis importez `templates` :

```
python
```

```
from fastapi import Depends, FastAPI, Request
from fastapi.staticfiles import StaticFiles
from sqlalchemy import select
```

```
from sqlalchemy.orm import Session

from app.core.config import templates
from app.core.database import Base, engine, get_db
from app.models.article import Article

# ... reste du fichier
```

Les vues

Maintenant que nous avons rangé notre modèle à sa place, nous allons faire de même avec les vues, aka les fonctions qui construisent le résultat envoyé au navigateur, que ça soit du HTML ou du JSON.

Pour pouvoir mettre les vues autre part que dans le fichier `main.py`, FastAPI utilise le concept de **routeur**. Ce routeur va faire le lien entre votre app principale FastAPI et des vues placées dans des modules/fichiers Python.

Dans votre répertoire `app/views`, créez un fichier nommé `article.py` qui contiendra vos vues. Placez-y le code suivant :

python

```
# app/views/article.py

from fastapi import APIRouter, Depends, Request
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.config import templates
from app.core.database import get_db
from app.models.article import Article

articles_views = APIRouter()

@articles_views.get("/articles/create", include_in_schema=False)
async def articles_create(request: Request, db: Session = Depends(get_db)):
    article = Article(
        title="Mon titre de test", content="Un peu de contenu<br />avec
```

```
deux lignes"
    )
    db.add(article)
    db.commit()
    db.refresh(article)

    return templates.TemplateResponse(
        request, "articles_create.html", {"article": article}
    )

@articles_views.get("/articles", include_in_schema=False)
async def articles_list(request: Request, db: Session = Depends(get_db)):
    articles_statement = select(Article).order_by(Article.created_at)
    articles = db.scalars(articles_statement).all()

    return templates.TemplateResponse(
        request, "articles_list.html", {"articles": articles}
    )

@articles_views.get("/api/articles")
async def api_articles_list(db: Session = Depends(get_db)):
    articles_statement = select(Article).order_by(Article.created_at)

    return db.scalars(articles_statement).all()
```

Nous avons sélectionné les imports dont nous allons avoir besoin et créons un objet `articles_views` de type [APIRouter](#) qui va nous permettre d'y attacher nos vues. Vous noterez que la seule chose que nous avons changé est le décorateur situé avant chaque fonction. Au lieu d'appeler `app.get` vous appelons maintenant `articles_views.get`.

Nous allons maintenant devoir faire le ménage dans `main.py` et appeler la fonction `app.include_router` pour inclure le routeur que nous venons de déclarer. Voici à quoi devrait ressembler votre fichier `main.py` :

```
python
```

```
# app/main.py
```

```
from fastapi import FastAPI, Request
```

```
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from app.core.database import Base, engine
from app.views.article import articles_views

app = FastAPI()

app.mount("/public", StaticFiles(directory="public"), name="public")

templates = Jinja2Templates(directory="app/templates")

Base.metadata.create_all(bind=engine)

app.include_router(articles_views, tags=["Articles"])

@app.get("/", include_in_schema=False)
async def root(request: Request):
    return templates.TemplateResponse(request, "home.html")
```

Amélioration de la configuration

Pour finir cette partie sur la restructuration du code, nous allons voir comment créer une classe de Settings qui va nous permettre de regrouper toutes nos valeurs de configuration au même endroit et nous donner par la suite plus de flexibilité (notamment grâce à l'utilisation de variables d'environnement). Nous allons par exemple y mettre l'url d'accès à la base de données, l'emplacement des templates ou encore celui des fichiers publics. Même si notre projet n'est pas très complexe pour l'instant, c'est une bonne pratique qui pourra vous faire gagner du temps à l'avenir.

Commencez par installer le package requis :

```
bash
```

```
(venv) $ pip install pydantic-settings
```

Puis modifiez le contenu du fichier `config.py` dans votre répertoire `app/core/` et mettez-y le contenu qui suit :

```
python
```

```
# app/core/config.py

import os

from fastapi.templating import Jinja2Templates
from pydantic_settings import BaseSettings

dir_path = os.path.dirname(os.path.realpath(__file__))

class Settings(BaseSettings):
    APP_NAME: str = "fastapi-tutorial"
    APP_VERSION: str = "0.0.1"
    SQLITE_URL: str = "sqlite:///./sql_app.db"

    TEMPLATES_DIR: str = os.path.join(dir_path, "..", "templates")
    STATIC_FILES_DIR: str = os.path.join(dir_path, "..", "..", "public")

settings = Settings()

templates = Jinja2Templates(directory=settings.TEMPLATES_DIR)
```

Il vous faut maintenant modifier vos fichiers `main.py` et `database.py` pour utiliser les valeurs de votre configuration plutôt que celles qui étaient codées en dur.

Pour `main.py` il s'agit de modifier le paramètre `directory` de `StaticFiles` en important `settings` et en utilisant la valeur `STATIC_FILES_DIR` :

```
python
```

```
# app/main.py

from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
```



```
from app.core.config import settings
from app.core.database import Base, engine
from app.views.article import articles_views

app = FastAPI()

app.mount("/public", StaticFiles(directory=settings.STATIC_FILES_DIR),
name="public")

# ... reste du fichier
```

Pour `database.py` c'est l'emplacement de la base de données qui doit être changé en utilisant `SQLITE_URL` :

python

```
# app/core/database.py

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

from app.core.config import settings

engine = create_engine(settings.SQLITE_URL, connect_args=
{"check_same_thread": False})
```

Tests

L'écriture de tests est un sujet qui reviendra souvent dans le code que nous allons effectuer et pour cause : si vous voulez garantir la **qualité de votre code**, vous devez écrire des **tests automatisés** pour s'assurer qu'il fonctionne correctement.

Si vous ne savez pas ce que c'est, ce n'est rien de plus qu'un petit **robot/bout de code** qui va se charger d'appeler différentes parties de votre code et va s'assurer qu'il se comporte bien comme il devrait.

Si vous pensez que « c'est bon je peux tester tout seul à la main » ou encore que « mon code n'est pas si compliqué que ça, pas besoin de s'embêter » vous êtes soit très débutant et il serait bien de me croire sur parole, ou soit très expérimenté et là je ne peux plus rien pour vous 😊

Quoiqu'il en soit, lorsque que l'on utilise un langage comme Python où les erreurs sont détectées au *runtime*, c'est à dire lorsque l'application est lancée et utilisée, c'est une bonne idée de mettre toutes les chances de son côté pour détecter les bugs avant que l'application ne soit utilisée. Sinon, évidemment, vous laisserez le soin à vos utilisateurs de découvrir vos bugs.

Bon à savoir : il existe d'autres langages de programmation que l'on appelle *statiquement typés* et qui disposent d'un compilateur qui vérifiera une partie des erreurs avant même que vous puissiez lancer l'application (*Rust, Java,...*). Mais comme ce n'est pas le cas de Python, écrire des tests est encore plus important. À noter que même si ces langages disposent d'un compilateur, il ne dispense pas d'écrire des tests mais va nous en économiser quelques uns.

Le logiciel qui va nous permettre de tester notre programme est appelé **pytest**. Commençons par l'installer dans notre virtualenv :

code

```
(venv) $ pip install pytest
```

Nous allons commencer par effectuer un test tout simple : s'assurer que notre page d'accueil / se charge sans erreur.

Test de la page d'accueil

Nous allons commencer par écrire le test qui va charger la page d'accueil. Si ce n'est pas déjà fait assurez-vous d'avoir créé le répertoire `app/tests/` et d'y avoir mis un fichier vide nommé `__init__.py`.

Créez ensuite un répertoire où nous allons mettre les tests de nos vues : `app/tests/views/`, placez-y un fichier `__init__.py` vide.

Dans `app/tests/views/` créez un fichier nommé `test_home.py` avec le contenu suivant :

python

```
# app/tests/views/test_home.py

from fastapi.testclient import TestClient

def test_home(client: TestClient) -> None:

    response = client.get("/")
    assert response.status_code == 200
    assert "Current url" in response.text
```

Ici, nous utilisons un "faux navigateur/client web" nommé `TestClient` qui va nous permettre d'aller charger des pages comme nous le ferions avec Chrome ou Firefox de manière presque classique (avec l'exécution du Javascript en moins, entre autres).

Nous demandons à notre client d'aller charger la page à l'adresse `/`. Nous utilisons ensuite un mot clé dédié aux tests : **`assert`**. Ce mot clé nous permet de nous assurer que la condition qui suit est vraie. Dans notre cas, nous commençons par vérifier que le *status code* HTTP renvoyé par FastAPI est bien 200.

Lors d'une requête HTTP sur le web, le serveur renvoie **toujours** un code de statut pour nous informer du déroulement de la requête. Le code 200 nous indique que tout s'est déroulé sans erreur.

Bon à savoir : Si le code de statut commence par 5, c'est une erreur du côté du serveur, donc vous ne pouvez a priori rien y faire (500 par exemple)

Si le code de statut commence par 4, c'est une erreur du côté du client HTTP (donc vous) : 404 page non trouvée (mauvaise adresse), 403 accès refusé (besoin de se connecter), ...

Si le code de statut commence par 3, le serveur indique une redirection (changement d'adresse)

Si le code de statut commence par **2**, c'est que tout va bien

Si le code de statut commence par **1**, cela indique juste la réception d'une information

Vous trouverez plus de détail sur la [page wikipedia dédiée](#)

Vous noterez que l'on teste aussi le contenu de la page HTML en s'assurant que le contenu qui nous est renvoyé contient bien la phrase `Current url`.

Pour pouvoir lancer ce test, il va nous manquer un fichier qui nous permet de configurer pytest et notamment qui va nous permettre de créer les objets qui sont nécessaires à l'exécution des tests.

Créez un fichier nommé `conftest.py` dans `app/tests/` et placez-y le contenu suivant :

python

```
# app/tests/conftest.py
import pytest
from fastapi.testclient import TestClient

from ..main import app

# Test client fixture
@pytest.fixture()
def client() -> TestClient:
    return TestClient(app)
```

Nous écrivons ici une fonction `client` qui retourne un client HTTP initialisé avec notre application FastAPI. C'est ce client qui va nous permettre de faire des appels à nos vues et d'en récupérer le contenu des réponses.

Vous pouvez noter l'utilisation du décorateur python

python

```
@pytest.fixture()
```

Le fait d'entourer notre fonction par ce décorateur va nous permettre d'obtenir le résultat de la fonction `client()` directement dans notre fonction de test `test_home`.

python

```
# app/tests/views/test_home.py

# ... début du fichier

def test_home(client: TestClient) -> None:
```

En effet, dans notre fichier de test, le seul fait d'ajouter un paramètre nommé `client` à notre fonction va suffire à `pytest` pour aller chercher, dans le fichier `conftest.py` une fonction décorée avec `@pytest.fixture()` correspondant au même nom. Ici, puisque nous déclarons un paramètre nommé `client`, `pytest` va aller automatiquement chercher une fonction nommée `client()` dans le fichier `conftest`.

Vous devriez maintenant être capable de lancer votre test avec la ligne de commande suivante à la racine du projet :

code

```
(venv) $ pytest app/tests/views/test_home.py
```

Et vous devriez voir un résultat de ce genre s'afficher :

code

```
===== test session starts
=====
platform linux -- Python 3.11.5, pytest-8.0.0, pluggy-1.4.0
rootdir: /home/vjousse/usr/src/python/fastapi-beginners-guide
plugins: anyio-4.2.0
collected 1 item

app/tests/views/test_home.py .
[100%]

===== 1 passed in 0.02s
```

=====

Alors certes nous n'avons pour l'instant qu'une vue à tester mais, croyez-moi, lorsque vous en aurez des dizaines voire des centaines, les tester manuellement ne sera même plus envisageable.

Test des articles

Créez un fichier nommé `test_articles.py` dans le répertoire `app/tests/views/` et placez-y le contenu suivant :

python

```
# app/tests/views/test_articles.py

from fastapi.testclient import TestClient

from app.models.article import Article
from app.tests.conftest import TestingSessionLocal

def test_create_article(client: TestClient, session: TestingSessionLocal) -
> None:
    article = Article(
        title="Mon titre de test", content="Un peu de contenu<br />avec
deux lignes"
    )

    session.add(article)
    session.commit()

    response = client.get("api/articles")
    assert response.status_code == 200
    content = response.json()
    assert len(content) == 1
    first_article = content[0]
    assert first_article["title"] == article.title
    assert first_article["content"] == article.content
```

Dans ce test nous créons un article dans la base de données et nous vérifions ensuite qu'il est bien affiché dans le json de la page `api/articles`. Rien de bien sorcier ici, si ce n'est la variable `session` passée en paramètre, variable qui nous donne accès à la base de donnée. Si vous avez bien suivi ce que je vous ai dit plus haut, cette variable doit venir d'une fixture configurée dans `conftest.py`. Mettez à jour votre fichier `app/tests/conftest.py` de la manière suivante :

```
python
```

```
# app/tests/conftest.py

from typing import Iterator

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool

from ..core.database import Base, get_db
from ..main import app

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app_test.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)

TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)

@pytest.fixture()
def session() -> Iterator[TestingSessionLocal]:
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)

    db = TestingSessionLocal()
```

```
try:
    yield db
finally:
    db.close()

# Test client
@pytest.fixture()
def client(session: TestingSessionLocal) -> Iterator[TestClient]:
    # Dependency override

    def override_get_db():
        try:
            yield session
        finally:
            session.close()

    app.dependency_overrides[get_db] = override_get_db

    yield TestClient(app)
```

On voit que l'on a rajouté une fixture nommée `session` qui prend en charge la création de la base de données avec les paramètres définis plus haut. Une partie de ce code devrait vous être familier : il est quasiment identique au code utilisé dans notre appli FastAPI précédemment.

Notez aussi que nous avons mis à jour la fixture `client`. Cette fixture prend maintenant un paramètre, `session`, qui est en fait la fixture que nous venons de créer (c'est Pytest qui s'autodébrouille pour la créer et l'injecter comme paramètre à notre fonction). Cela va nous permettre d'avoir accès à la base de données dans le code de `client`. FastAPI dispose d'une méthode `dependency_overrides` qui va nous permettre de surcharger une dépendance. Rappelez-vous, dans les vues de nos articles, nous avions une ligne comme celle-ci :

python

```
# app/views/article.py
```

```
# ... début du fichier
```



```
async def articles_create(request: Request, db: Session = Depends(get_db)):
```

```
# ... suite du fichier
```

Cette ligne nous informe que la fonction prend en paramètre une dépendance, `db` qui doit être récupérée en appelant la fonction `get_db`.

Dans notre fixture `client`, la ligne :

```
python
```

```
app.dependency_overrides[get_db] = override_get_db
```

vient signifier à FastAPI qu'il doit remplacer la fonction `get_db` dans toutes les dépendances par la fonction `override_get_db`. Cette dernière renvoyant la base de données de test, cela va avoir pour effet de faire tourner notre application FastAPI sur la base de données de test, au lieu de la base de données par défaut.

Ensuite, lancez votre test comme précédemment :

```
code
```

```
(venv) $ pytest app/tests/views/test_articles.py
```

Vous devriez obtenir quelque chose de ce style :

```
code
```

```
===== test session starts
=====
platform linux -- Python 3.11.5, pytest-8.0.0, pluggy-1.4.0
rootdir: /home/vjousse/usr/src/python/fastapi-beginners-guide
plugins: anyio-4.2.0
collected 1 item

app/tests/views/test_articles.py .
[100%]

===== 1 passed in 0.09s
=====
```

Vous pouvez lancer tous vos tests en même temps en ne spécifiant que le répertoire des tests à pytest :

code

```
(venv) $ pytest app/tests/
```

Vos deux fichiers de tests sont alors mentionnés dans le rapport :

code

```
===== test session starts
=====
platform linux -- Python 3.11.5, pytest-8.0.0, pluggy-1.4.0
rootdir: /home/vjousse/usr/src/python/fastapi-beginners-guide
plugins: anyio-4.2.0
collected 2 items

app/tests/views/test_articles.py .
[ 50%]
app/tests/views/test_home.py .
[100%]

===== 2 passed in 0.16s
=====
```

Conclusion

Nous venons d'achever une étape qui peut paraître fastidieuse mais qui est haut combien importante : le refactoring et le test de notre code. Cette étape nous permet maintenant de partir sur des bases propres et solides pour ajouter des fonctionnalités à notre application.

Comme d'habitude, le code pour cette partie est [accessible directement sur Github](#).

Pour la partie 4, c'est par ici : [création, récupération et suppression des articles](#).

 Me contacter sur [Mastodon](#)

 Me contacter par [mail](#)

 S'abonner au flux [RSS](#)

 Recevoir les nouveaux articles par mail

(votre adresse ne sera jamais partagée à des tiers)

 Contenu fourni sous license [CC-By Licence](#).

 Construit avec [Rust](#), code source disponible sur [Github](#).

 Mettez-vous à Vim avec [mon livre à prix libre](#).